Kick Start 2018 - Round E

Analysis: Board Game

Board Game: Analysis

Small dataset

Bahu needs to determine the best possible card distribution given that Bala will choose a distribution uniformly at random. In the Small dataset, **N** = 3, so there are only 9! / 3! / 3! / 3! = 1680 different ways for a player to distribute their cards. (If there are multiple cards with the same strength value, some of these distributions may be practically equivalent, but we will treat them as different for simplicity.) We can enumerate all 1680 possible distributions U_i for Bahu, and all 1680 possible distributions A_j for Bala. (These variables are named after the last letters of the players' names.) For each U_i, we find the fraction of all A_js that lose against that U_i. The largest such fraction is our answer. The time complexity is on the order of 1680^2 times a small constant factor.

Large dataset

In the Large dataset, it is possible that **N** = 5. In that case, there are 15! / 5! / 5! / 5! = 756756 different distributions. We cannot use the above strategy with a 756756^2 time factor.

Only the sums of the cards in the three battlefields matter; let them be U1, U2, U3 for Bahu and A1, A2, A3 for Bala. Then Bahu wins if at least two of the following inequalities are satisfied: U1 > A1, U2 > A2, U3 > A3.

We can deal with the "at least two" part of that criterion by using the inclusion-exclusion principle. Then, for each U_i:

The number of A_js satisfying the above criterion =
The number of A_js satisfying U1 > A1 and U2 > A2 +
The number of A_js satisfying U1 > A1 and U3 > A3 +
The number of A_js satisfying U2 > A2 and U3 > A3 -
2 × the number of A_js satisfying U1 > A1 and U2 > A2 and U3 > A3.

The last of those quantities is the most difficult one to calculate. Here we need another observation that there are only 15 choose 5 = 3003 different possibilities for all U1, U2, U3. We can label these possibilities 1 through 3003.

This is a 3-dimensional query, but we can remove 1 dimension by processing the Us in increasing order of U1 and also preprocessing As in increasing order of A1. After that, we can create a 2-dimensional segment tree to store all (A2, A3)s that have A1 < U1. We can find all (A2, A3)s such that U2 > A2 and U3 > A3 in log 3003 × log 3003 time complexity. The other 3 quantities in the equation above can also be found in a similar way. The overall time complexity is on the order of 756756 × log 3003 × log 3003 times a small constant factor, which is fast enough to solve this dataset.

Kick Start 2018 - Round E

Analysis: Milk Tea

For the Small dataset, there are only $2^{\mathbf{P}}$ different combinations of options available. Since **P** is at most 10, there are only 1024 possible combinations. Take the combination that gives the fewest mistakes that isn't forbidden.

For the Large dataset, first notice that there are at most 100 forbidden combinations. One approach is to generate the 101 combinations that cause the fewest complaints and take the best one that is not forbidden (there is at least one combination that is not forbidden in the top 101).

To generate these combinations, begin by noticing that in terms of the number of complaints you will get, each of the options can be considered separately. To help us later, preprocess for each option, how many complaints we would get if we were to get the milk tea with that option and without it. We will now build up the best combinations one option at a time.

Let T_k denote the top 101 combinations that generate the fewest complaints when we consider only the first k options, represented as a binary string (tiebreaking arbitrarily). The key idea is to note that each combination in T_{k+1} has a combination from T_k as a prefix.

This is easy to show by contradiction. Take any string S from T_{k+1} and remove the last bit to get the prefix S'. Suppose for a contradiction that S' is not T_k. Taking any string in T_k and appending the removed bit will give a combination that generates strictly fewer (when considering the tiebreak) complaints. This gives 101 strings that generate fewer complaints, which contradicts S being in T_{k+1}.

So the final algorithm is as follows:

1. T_0 is the empty set.
2. To generate T_k (for $k > 0$), take each string in T_{k-1} and try appending both a 0 and a 1. This will give at most 202 potential answers, of which we keep the best 101 (in general, we keep the top **M**+1).
3. Take the best combination from $T_{\mathbf{P}}$ that isn't forbidden.

Naively, this can be done in $O(\mathbf{P}^2\mathbf{M})$, but can also be done faster in $O(\mathbf{PM})$. In total, the algorithm takes $O(\mathbf{PN} + \mathbf{P}^2\mathbf{M})$ or $O(\mathbf{PN} + \mathbf{PM})$.

Kick Start 2018 - Round E

Analysis: Yogurt

Yogurt: Analysis

Intuitively, if Lucy wants to maximize the amount of yogurt she consumes, she should consume as many cups as possible each day, and always choose a cup that is closest to expiring.

Small dataset

We can directly implement the above strategy to solve the Small dataset, in which Lucy can only consume one cup per day. On each day, we scan the list for the smallest unselected cup that has not expired, consume it, and remove it from the list. Since we make up to **N** passes through a list with up to **N** items, the time complexity is O($\mathbf{N}^2$).

Large dataset

To improve the above strategy to solve the Large dataset, we can first sort the cups in non-decreasing order of time until expiration. Then, on each day, we remove all expired cups from the beginning of the list and then consume the next **K**. Since we only handle each cup in the sorted list once, the initial sorting step determines the overall time complexity, which is O(**N** log **N**).

But we can do even better! Notice that there are only **N** cups, and since **K** ≥ 1, Lucy could consume or discard all of them in at most **N** days. So any $\mathbf{A_j} \geq \mathbf{N}$ can be replaced with **N**. Then, we can count how may cups will expire on each day, and then proceed backwards from the **N**th day to the first day. On each day, Lucy consumes at most **K** cups, and moves any remaining cups from that day to the previous day, since it is safe to consume them earlier. (Any extra cups left over on day 1 must be discarded.) We make one forward pass through the data to count the cups and one reverse pass to answer the question, so this strategy is O(**N**).